

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

TECHNICAL REPORT

Linux Process Telemetry and Unix-Domain IPC Services

AUTHOR

Reza Sanaee Nezhad

Student ID: 401101502

INSTRUCTOR / INSTRUCTORS

Dr Iman Gholampour

ACADEMIC CONTEXT

Embedded Systems

Contents

1 Startup chain 2

2 Host/target compilation and toolchains 2

3 Root directories and filesystem choices 2

4 Device Tree and overlays 3

5 Timing predictability 3

6 Unix-domain request/response implementation 3

7 procwatch code organization and build 5

8 System telemetry and CPU calculation 7

9 Per-process activity, names, and RSS 9

10 Dynamic /proc races 10

11 Configuration, graceful termination, and restart tests 10

1 Startup chain

The bootloader initializes required processor, RAM, and clock resources, obtains the kernel image and hardware description from nonvolatile storage, establishes boot arguments, and transfers execution to the kernel. Platform ROM or earlier boot stages may precede this loader; it is not universally the first code executed after power-on. The kernel manages processors, memory, devices, scheduling, and networking. It initializes its own structures and drivers, mounts an initial/root filesystem, and starts the first userspace program as PID 1. The root filesystem supplies executables, configuration, libraries, and the directory structure needed by userspace. It is data and storage organization, not an executable that calls the next stage. Missing required init/root contents can prevent startup, although an initramfs may provide an earlier userspace environment. A C library such as glibc, musl, or uClibc supplies standard operations and system-call wrappers. Dynamically linked applications require a compatible runtime library and loader; static linkage changes that requirement. C++ streams additionally rely on the C++ runtime, rather than being provided by libc alone.

init/systemd configures userspace, mounts virtual filesystems, manages services, and handles PID-1 responsibilities. It can configure networking and launch daemons before the application. User applications perform the device's final function, such as control, a GUI, or data processing, and are commonly started through service definitions or init scripts.

2 Host/target compilation and toolchains

A normal x86/x86-64 compiler emits instructions for its configured target, which an ARM processor does not execute natively. Moving that binary to the board commonly produces an executable-format failure. Cross compilation builds a binary for the board on the development machine, for example with an appropriate ARM Linux g++ toolchain. Matching instruction-set architecture is necessary but ABI, operating-system interface, word size, runtime libraries, and floating-point ABI also matter. In this scenario the laptop is the development/build machine and the board is the execution target. Toolchain terminology can distinguish build, host, and target in more complex compiler-building situations. Native compilation uses a compatible local execution target; cross compilation selects another target configuration. The compiler translates C++ into target code. The assembler produces object files. The linker combines objects and target-compatible libraries. Kernel userspace headers define interface structures and constants; target C/C++ libraries provide the runtime. GDB with gdbserver can support remote debugging. Make tracks dependencies and avoids unnecessary recompilation, while CMake generates build descriptions and centralizes configuration. A CMake toolchain file can select cross-compilers and a target sysroot without rewriting application source.

3 Root directories and filesystem choices

/bin conventionally supplies essential commands such as sh, ls, and cp, although merged-/usr systems organize these through links. /etc contains system and service configuration. /lib supplies runtime libraries and, in conventional layouts, kernel modules. /dev contains device interfaces, which do not correspond one-to-one with every physical component. /var holds changing application/system state such as logs.

/proc exposes process and kernel information. /sys exposes the kernel's device/driver model and related attributes. Their content is generated through virtual filesystem interfaces; it is not an ordinary static set of files stored on flash. Reading these interfaces can change between operations, which is central to the process-monitoring exercise.

ext4 supports writable persistent data and journals metadata to assist recovery. Journaling does not guarantee that every recent application write survives power loss; synchronization and device be-

havior matter. Frequent writes can contribute to flash wear. SquashFS is compressed and read-only, useful for immutable system images; it avoids writes to that filesystem during ordinary operation but does not make the entire storage device immune to failures. tmpfs is temporary memory-backed storage; its data does not survive a reboot, and it can be swapped depending on system configuration. Reducing flash-backed temporary writes through tmpfs is useful only with appropriate memory/swap policy.

4 Device Tree and overlays

Device Tree separates board hardware description from kernel source and is especially useful for devices that cannot self-enumerate. It can describe CPU topology, memory ranges, peripheral register addresses, interrupt routing, clocks, regulators, and pin configuration. Compatible descriptions and available drivers allow one kernel build to support multiple boards, but not arbitrary hardware without matching support. An overlay modifies a base tree, for example enabling an expansion board or describing an added peripheral. The example selects i2c1, sets status to okay, and adds a BMP280 node at address 0x76 with compatible bosch,bmp280. A bootloader or supported runtime overlay mechanism can apply it without rebuilding kernel code if the corresponding driver is already available. Merely supplying an overlay cannot create a missing driver or guarantee runtime reconfiguration on every platform.

5 Timing predictability

Standard Linux workloads can experience scheduling delay, nonpreemptible critical sections, interrupt-disabled intervals, memory faults, reclaim/storage work, and driver/hardware interference. Kernel versions and preemption settings affect which sections can be preempted. These sources make a hard deadline guarantee a system-analysis problem rather than a label attached to a distribution. PREEMPT_RT increases preemptibility and threads many interrupt handlers, helping reduce latency while preserving Linux libraries, drivers, and development tools. Synchronization changes and configuration can affect throughput. The experiment report characterizes it as only soft/firm real time and quotes a universal microsecond scale; those conclusions are qualified. Suitability depends on measured worst-case behavior, the platform, workload, and required bound. A split design assigns time-critical work to a microcontroller or real-time processor with an RTOS, while Linux handles networking, UI, and data. Cortex-M/Cortex-R and Cortex-A illustrate these roles. It introduces two toolchains/codebases and a communication interface, perhaps shared memory or OpenAMP. It can improve isolation, but does not confer absolute zero-jitter operation or immunity to shared power, memory, or bus faults. The recorded examples of motor control, medical functions, and high-rate signal paths motivate this split; their actual assurance depends on requirements and verified design.

6 Unix-domain request/response implementation

The C++ applications communicate through AF_UNIX at /tmp/hw1_socket. The server removes a stale pathname with unlink, creates/binds a socket, listens, and repeatedly accepts clients. It reads a command, sends the response, closes the client connection, and returns to accept. TIME uses date/time library functions; ECHO returns the text following the command. The client combines argc/argv into a request, connects, sends, reads a response, prints it, and exits. The supplied Makefile builds server and client with g++ using C++11 and supplies a clean target. In the recorded test, TIME returned Mon Jul 6 17:33:03 2026. ECHO returned Hello Embedded Systems! in a second connection. Both terminal captures are preserved. These tests establish two successive requests in a local demonstration. They do not establish unlimited message length, complete partial-read/

write handling, concurrent-client throughput, or access control. Socket-directory permissions, safe handling of stale paths, framing, maximum request size, malformed commands, and orderly cleanup are relevant extension points.



Figure 1: Diagram or recorded result: Unix-domain request/response implementation.

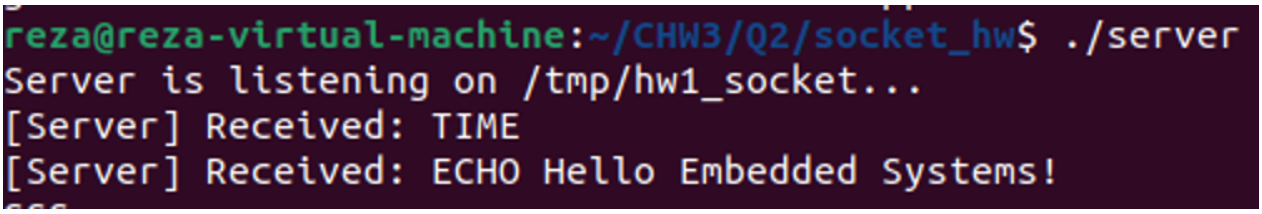


Figure 2: Diagram or recorded result: Unix-domain request/response implementation.

7 procwatch code organization and build

The monitor separates main/sysinfo implementation under src and sysinfo.h under include. Both Make and CMake are supplied. Later phases add process enumeration and background-service configuration while retaining the module separation.

```
reza@reza-virtual-machine:~/CHW3/Q2/socket_hw$ ./client
Usage: ./client <Command> [Arguments]
Commands:
  TIME
  ECHO <text>
reza@reza-virtual-machine:~/CHW3/Q2/socket_hw$ ./client TIME
Mon Jul  6 17:33:03 2026
reza@reza-virtual-machine:~/CHW3/Q2/socket_hw$ ./client ECHO Hello Embedded Systems!
Hello Embedded Systems!
```

Figure 3: Diagram or recorded result: procwatch code organization and build.

```
reza@reza-virtual-machine:~/CHW3/Q3/P1/procwatch$ make clean
rm -rf obj procwatch
reza@reza-virtual-machine:~/CHW3/Q3/P1/procwatch$ mkdir build && cd build
reza@reza-virtual-machine:~/CHW3/Q3/P1/procwatch/build$ ..
..: command not found
reza@reza-virtual-machine:~/CHW3/Q3/P1/procwatch/build$ .. make
..: command not found
reza@reza-virtual-machine:~/CHW3/Q3/P1/procwatch/build$ cmake ..
```

Figure 4: Diagram or recorded result: procwatch code organization and build.

```
reza@reza-virtual-machine:~/CHW3/Q3/P1/procwatch/build$ cmake ..
-- The C compiler identification is GNU 11.4.0
-- The CXX compiler identification is GNU 11.4.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/reza/CHW3/Q3/P1/procwatch/build
```

Figure 5: Implementation and output: procwatch code organization and build.

```
================ LOCAL SYSTEM MONITOR ================
Kernel Version:          6.8.0-124-generic
CPU Cores:               4
Load Average:            0.41, 0.14, 0.12
Overall CPU Usage:       25.41 %
------------------------------------------------------
MEMORY STATUS (KB):
Total           Used            Free            Available
6019536         1462400         1807620         4557136
======================================================
Press Ctrl+C to exit.
```

Figure 6: Diagram or recorded result: procwatch code organization and build.

```
reza@reza-virtual-machine:~/CHW3/Q3/P1/procwatch$ make
mkdir -p obj
g++ -Wall -std=c++11 -Iinclude -c src/main.cpp -o obj/main.o
g++ -Wall -std=c++11 -Iinclude -c src/sysinfo.cpp -o obj/sysinfo.o
g++ -Wall -std=c++11 -Iinclude -o procwatch obj/main.o obj/sysinfo.o
reza@reza-virtual-machine:~/CHW3/Q3/P1/procwatch$ ./procwatch interactive

================ LOCAL SYSTEM MONITOR ================
Kernel Version:          6.8.0-124-generic
CPU Cores:               4
Load Average:            0.31, 0.15, 0.11
Overall CPU Usage:       1.63 %
------------------------------------------------------
MEMORY STATUS (KB):
Total           Used            Free            Available
6019536         1472096         1797576         4547440
======================================================
Press Ctrl+C to exit.
```

Figure 7: Implementation and output: procwatch code organization and build.

8 System telemetry and CPU calculation

The interactive table reads kernel version from /proc/version, logical processor count from processor entries in /proc/cpuinfo, memory from /proc/meminfo, and one-, five-, and fifteen-minute load averages from /proc/loadavg. The recorded kernel is 6.8.0-124-generic. Used memory is estimated as MemTotal-MemAvailable; this is an available-capacity convention rather than a unique definition of all used memory. Load average is not the same as CPU percentage. The aggregate cpu line in /proc/stat contains cumulative user, nice, system, idle, iowait, irq, softirq, steal, and related counters. The code treats idle+iowait as idle and sums the selected busy counters separately. With consecutive samples, utilization is 100 times (DeltaTotal-DeltaIdle)/DeltaTotal. A single reading gives time accumulated since boot, not utilization over the last update. The interpretation of iowait and avoidance of double-counting guest fields should be explicit. At low activity the report records about 1.63 percent. With stress-ng –cpu 1 –timeout 30s on a four-logical-CPU system, htop shows one CPU near full use and procwatch reports 25.37 percent aggregate use. This is consistent with the expected approximately one-quarter aggregate busy time, allowing background activity and different sampling intervals. It is an informative cross-check rather than exact proof for every load pattern.

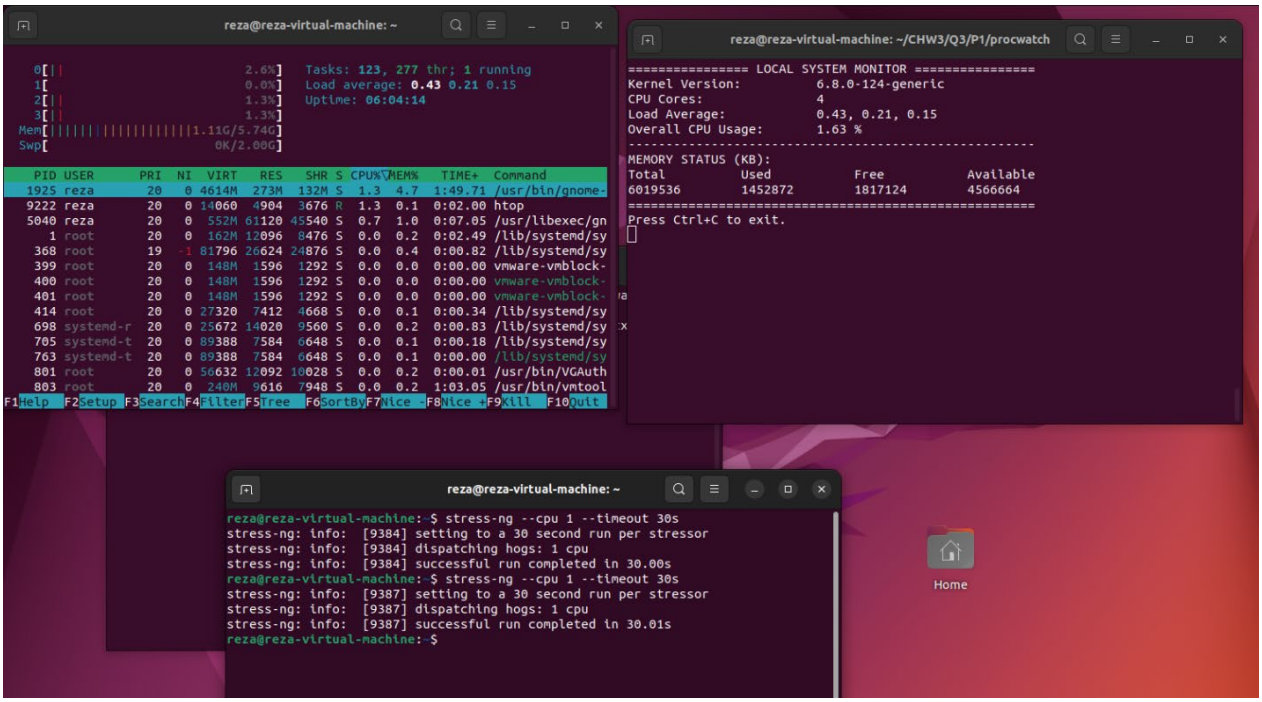


Figure 8: Diagram or recorded result: System telemetry and CPU calculation.

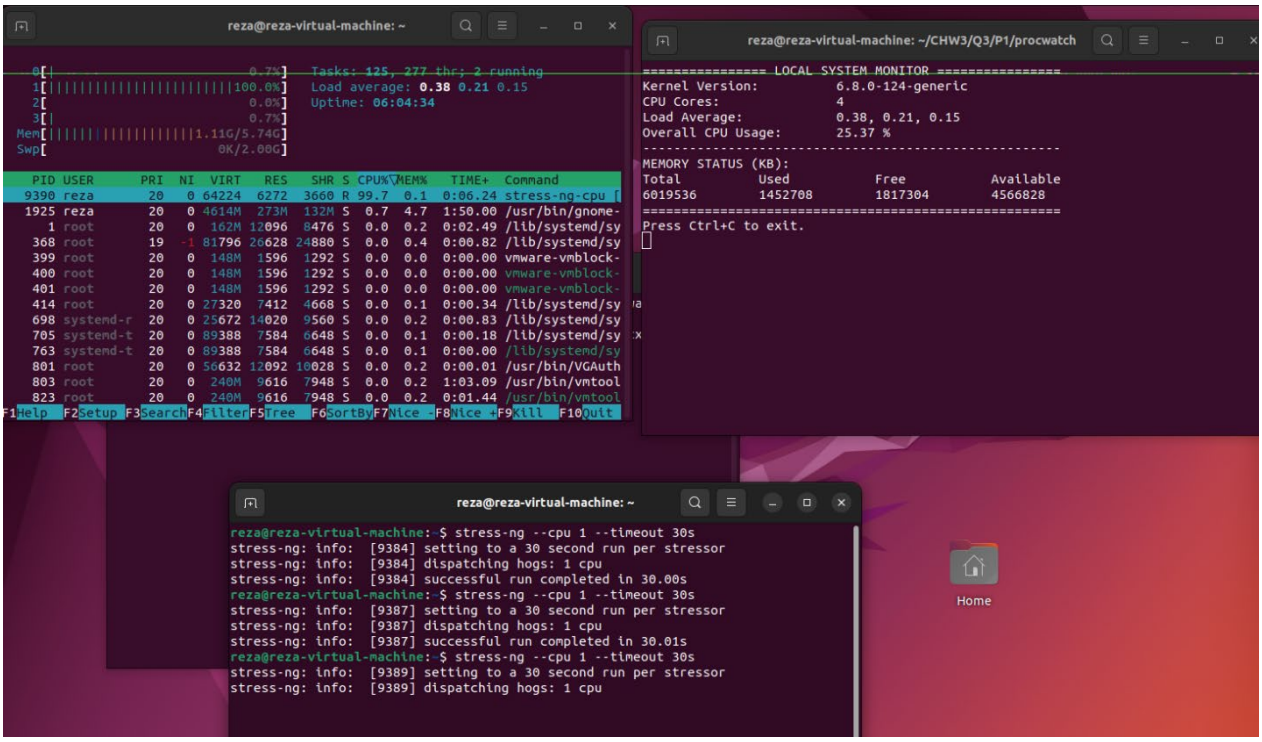


Figure 9: Diagram or recorded result: System telemetry and CPU calculation.

9 Per-process activity, names, and RSS

Numeric /proc directory names identify candidate PIDs. The process name is parsed between the first opening and final closing parenthesis of /proc/[pid]/stat, which matters because names can contain spaces. Fields utime and stime give user/system CPU time; their consecutive differences provide interval activity. Previous values are retained in a map. Dividing a process's tick difference by aggregate system tick difference gives a percentage of total multicore capacity. For the familiar one-core-normalized top-style percentage, divide by elapsed wall time times the clock-tick rate, or account for logical CPU count consistently. RSS is read from the second field of /proc/[pid]/statm in pages and converted using sysconf(_SC_PAGESIZE), then into KiB. Four-KiB pages are common but not assumed. RSS measures resident pages and does not simply equal privately owned memory. Candidates are sorted by activity and the selected top N shown.

```
================ LOCAL SYSTEM MONITOR ================
Kernel Version:          6.8.0-124-generic
CPU Cores:               4
Load Average:            0.15, 0.09, 0.08
Overall CPU Usage:       1.87 %
------------------------------------------------------
MEMORY STATUS (KB):
Total          Used           Free           Available
6019536        1430256        1837820        4589280
------------------------------------------------------
TOP 5 PROCESSES (By CPU Activity):
PID       NAME                CPU ACTIVITY   RSS (KB)
1925      gnome-shell         10             277456
9844      gnome-terminal-     4              52720
9882      procwatch           1              3916
803       vmtoolsd            1              9616
2065      ibus-daemon         1              13248
======================================================
Press Ctrl+C to exit.
```

Figure 10: Diagram or recorded result: Per-process activity, names, and RSS.

```
reza@reza-virtual-machine:~/CHW3/Q3/P2/procwatch$ make clean
rm -rf obj procwatch
reza@reza-virtual-machine:~/CHW3/Q3/P2/procwatch$ make
mkdir -p obj
g++ -Wall -std=c++11 -Iinclude -c src/main.cpp -o obj/main.o
g++ -Wall -std=c++11 -Iinclude -c src/sysinfo.cpp -o obj/sysinfo.o
g++ -Wall -std=c++11 -Iinclude -o procwatch obj/main.o obj/sysinfo.o
reza@reza-virtual-machine:~/CHW3/Q3/P2/procwatch$ ./procwatch interactive
```

Figure 11: Implementation and output: Per-process activity, names, and RSS.

10 Dynamic /proc races

A process can exit after enumeration but before its files are read. The program checks file-opening success and skips unavailable entries. This prevents the demonstrated missing-file path from crashing the monitor. Robust monitoring must also handle malformed/short reads, permission denial, PID reuse, changed counter baselines, and zero intervals. Skipping one common failure mode does not prove complete race freedom.

11 Configuration, graceful termination, and restart tests

The background configuration contains interval, n, and simulate_error, and its path is passed on the command line. A SIGTERM handler sets a control flag so the loop exits, a shutdown message is recorded, and the process returns zero. The deliberate simulate_error path exits with code one after several iterations. Signal handlers should restrict themselves to appropriate signal-safe operations; the ordinary loop should perform logging/cleanup.

procwatch.service selects background mode, uses Restart=on-failure, and sets RestartSec=5. In the ordinary test, a five-second interval and top-three output are recorded in the journal. In the fault test, the application logs CRITICAL: Simulated intentional error triggered! Exiting with code 1 and systemd restarts it approximately five seconds later. A successful SIGTERM termination and nonzero fault have different restart behavior. The evidence demonstrates these configured lifecycle paths, not resistance to every runtime failure.

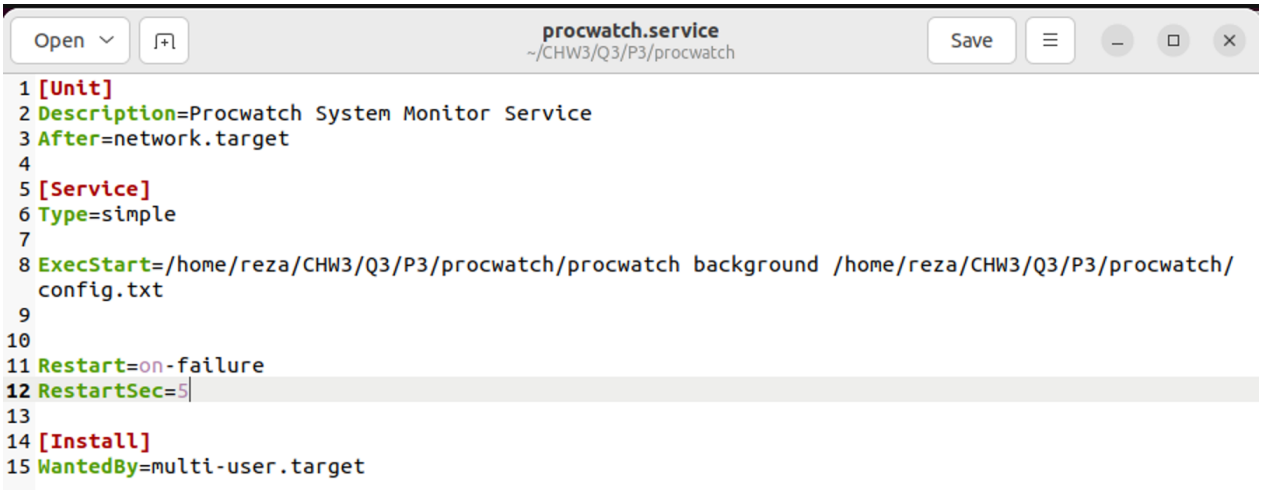


Figure 12: Diagram or recorded result: Configuration, graceful termination, and restart tests.

```
reza@reza-virtual-machine:~$ sudo systemctl start procwatch.service
reza@reza-virtual-machine:~$ journalctl -u procwatch.service -f
19:00:56 06 July   reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: proc
watch(PID:11080) kworker/u259:0-events_power_efficient(PID:8235) vmtoolsd(PID:803)
19:01:01 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: vmto
olsd(PID:803) procwatch(PID:11080) kworker/u259:3-events_freezable_power_(PID:10955)
19:01:06 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: vmto
olsd(PID:2156) kworker/0:2-events(PID:9982) kworker/2:2H-kblockd(PID:1848)
19:01:11 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: proc
watch(PID:11080) vmtoolsd(PID:2156) kworker/3:0H-kblockd(PID:2936)
19:01:16 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: gnom
e-shell(PID:1925) nautilus(PID:4791) Xwayland(PID:2283)
19:01:22 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: proc
watch(PID:11080) systemd-oomd(PID:7149) gvfs-udisks2-vo(PID:1967)
19:01:27 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: vmto
olsd(PID:803) vmtoolsd(PID:2156) procwatch(PID:11080)
19:01:32 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: naut
ilus(PID:4791) gnome-shell(PID:1925) gnome-terminal-(PID:10921)
19:01:37 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: gnom
e-shell(PID:1925) gnome-terminal-(PID:10921) vmtoolsd(PID:2156)
19:01:42 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: gnom
e-shell(PID:1925) procwatch(PID:11080) kworker/2:0-events(PID:10545)
19:01:47 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: gnom
e-shell(PID:1925) procwatch(PID:11080) gnome-terminal-(PID:10921)
19:01:52 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: gnom
e-shell(PID:1925) gnome-terminal-(PID:10921) vmtoolsd(PID:803)
19:01:57 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: gnom
e-shell(PID:1925) gnome-terminal-(PID:10921) procwatch(PID:11080)
19:02:02 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: gnom
e-shell(PID:1925) gnome-terminal-(PID:10921) vmtoolsd(PID:2156)
19:02:07 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: gnom
e-shell(PID:1925) gnome-terminal-(PID:10921) systemd-oomd(PID:7149)
19:02:12 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: proc
watch(PID:11080) kworker/u258:2-events_unbound(PID:9002) gnome-shell(PID:1925)
19:02:17 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: gnom
e-shell(PID:1925) gnome-terminal-(PID:10921) kworker/1:0-mpt_poll_0(PID:10096)
S19:02:22 06 July   reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: gno
me-shell(PID:1925) gnome-terminal-(PID:10921) procwatch(PID:11080)
19:02:27 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: gnom
e-shell(PID:1925) gnome-terminal-(PID:10921) vmtoolsd(PID:803)
19:02:32 06 جولای reza-virtual-machine procwatch[11080]: [Procwatch] Top 3 CPU Consumers: gnom
e-shell(PID:1925) gnome-terminal-(PID:10921) vmtoolsd(PID:2156)
```

Figure 13: Implementation and output: Configuration, graceful termination, and restart tests.

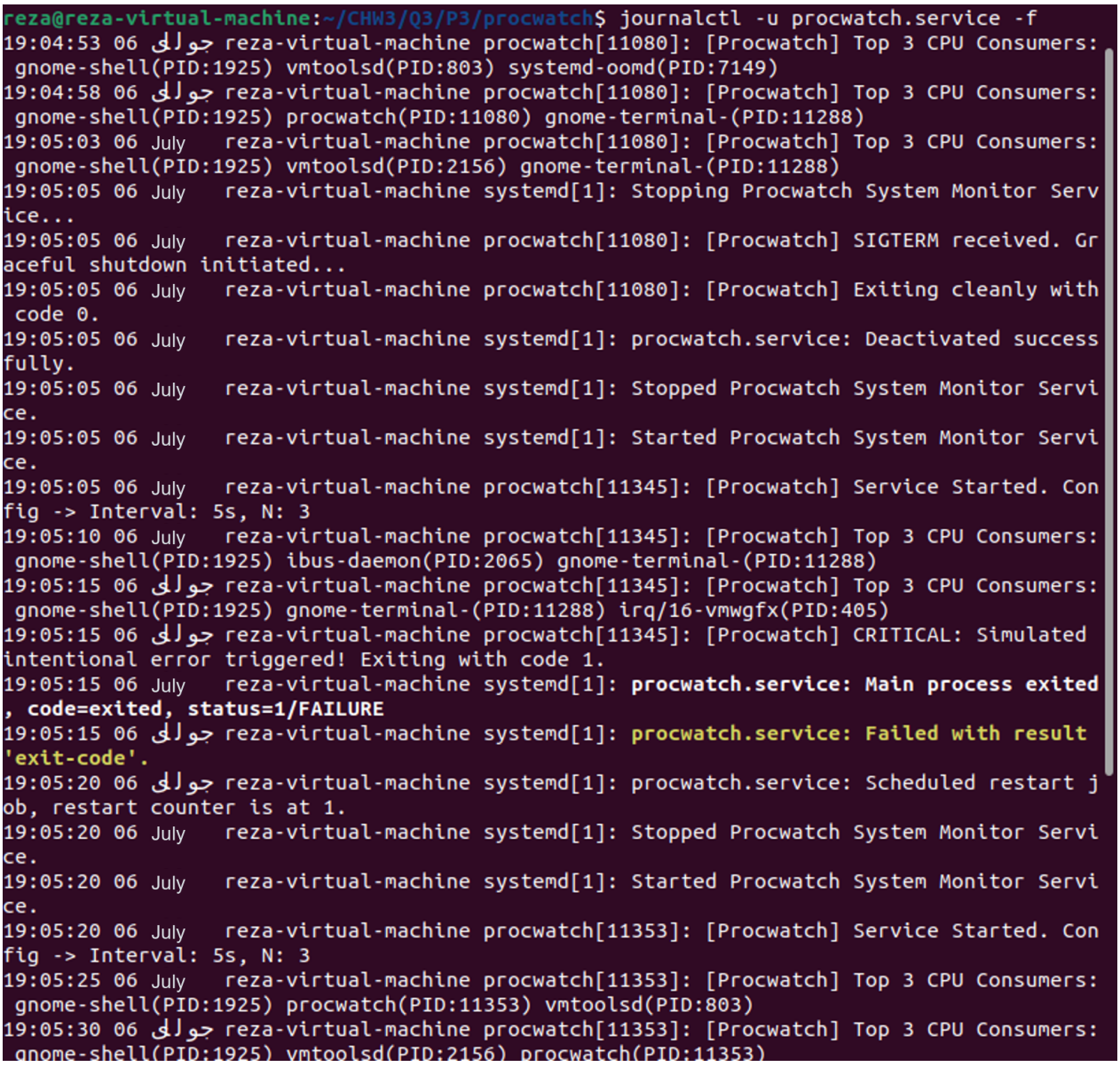


Figure 14: Diagram or recorded result: Configuration, graceful termination, and restart tests.